Alcovia Full Stack Engineering Intern

Time: 48 hours from when you receive this.

Stack: TypeScript, React Native (Expo - running on web is fine), Express, n8n.

Background
Alcovia is a study app for students in grades 6–12. Two of its core features:

- **Focus sessions** - a student starts a timer and tries to stay focused for a chosen duration. Finishing it earns rewards; bailing out doesn't.
- **Syllabus progress** - a student works through study tasks; completing them rolls up into per-subject progress.

Our students use the app on the move - metros, classrooms, patchy hostel Wi-Fi - and often on more than one device (e.g. a phone and a laptop). So these features have to keep working **with no network**, and stay **consistent across devices** once they reconnect. That's what this take-home is about.

What to build
An **offline-first** version of these two features, with a backend that keeps multiple devices in sync, and an n8n automation that reacts to what happens in the app.

Feature A - Focus sessions
A student starts a session with a chosen **target duration** (e.g. 25–120 minutes) and runs the timer.

- **Success:** the session reaches its **full target duration** while the student stays in the session.
- **Fail (abandoned):** the student taps **Give up**, **or** leaves the session / backgrounds the app for more than a short grace period (e.g. **5 seconds**) while the timer is running.
- On **success:** the student's **focus streak** advances, they earn **coins**, and the minutes count toward **today's focus total**.
- On **fail:** no reward; the attempt is recorded as failed with a reason (`give_up` or `app_switch`).
- All of this must work **fully offline** - a student can complete or fail sessions with no connection, and the results (and their effect on streak, coins, and today's total) sync up later.

Feature B - Syllabus progress
A student has a few **subjects**, each with **chapters**, each with **tasks**. A task has a status

(e.g. *Not started → In progress → Done*).

- Marking tasks updates progress: **chapter % = completed tasks ÷ total tasks**, and **subject % rolls up from its chapters**.
- Editing task status must work **offline** and update progress **instantly** on the device.

Feature C - The automation layer (n8n)
When a focus session is confirmed as a success on the server (after sync, counted once), the backend fires a webhook to an n8n workflow you build. That workflow sends a notification, for example a WhatsApp message: "Streak now 4 days, +50 coins."

- You may send to a real WhatsApp sandbox (AiSensy / Twilio / Meta) or to a mock notification sink (a second HTTP endpoint that just logs the payload). Your choice. The n8n workflow itself must be real and exported.
- The hard part, idempotency carries through to n8n: the same session can be replayed during sync, or arrive from both devices. The notification must be sent exactly once per successful session. Dedupe on a stable event/session id, not wall-clock time.

The thing that ties them together: two devices, one account
A student may have the app open on two devices. Both can go offline, both can make changes, and when they reconnect everything must **reconcile to one correct, identical state on both** - no lost edits, no duplicates, no rewards counted twice, and no notification fired twice.

Requirements (core)
Your solution must handle all of these correctly:

1. **Offline-first.** Every action - start / finish / fail a focus session, change a task's status - works instantly with no network, is stored **durably on the device**, and syncs when a connection returns.
2. **Two devices converge.** Run two clients on the same account; let them diverge offline; on reconnect they end in the **same state**.
3. **Idempotent rewards.** A focus session completed offline must count **exactly once** - retries or replays during sync must not award coins twice or bump the streak twice. After offline sessions from *both* devices sync, the **streak and today's focus total must be correct.**
4. **Idempotent automation. The n8n notification for a successful session must fire exactly once, even when the underlying event is replayed or arrives from both devices during sync.**

5. **Conflicts, resolved deliberately.** Decide and implement sensible behaviour for:

    - the **same task's status changed on both devices** (e.g. phone → *Done*, laptop → *In progress*),
    - a task **edited on one device and deleted on the other**,
    - the **same sync message arriving twice, or out of order**.

Pick a resolution strategy, implement it, and explain it. (A wall-clock "last write wins" won't behave well here - device clocks disagree.)

6. **Demonstrable.** Include a small dev panel to toggle each client **online/offline** and trigger the scenarios above, plus a view of each device's current state, so the behaviour can be shown end-to-end. The dev panel (or your logs) should also make the n8n notification firing exactly once visible.

Constraints

- **TypeScript + React Native (Expo) + Express + n8n. On-device** and server storage are your choice (IndexedDB / SQLite / AsyncStorage / a JSON file / in-memory are all fine).

- n8n can be n8n Cloud (free tier), self-hosted (npx n8n / Docker), whatever you like. WhatsApp delivery may be real or mocked, but the workflow must be genuine and exported as JSON in the repo.

- **No login** - hardcode a single studentId shared by both clients.
- Keep the UI **simple and functional** - this isn't a design task.
- Don't use an off-the-shelf "sync" product (e.g. Replicache / PowerSync / Firebase sync / Yjs as a black box). The sync and merge logic should be **your own** - you may implement a known algorithm yourself.

- **Note for web:** two browser tabs of the same site **share storage**. Give each client its **own storage** (separate browser profiles / incognito, or a per-client storage namespace) so they behave like two real devices.

- Where something isn't specified (exact thresholds, copy, schema), **make a reasonable choice and note it in your README.**

Deliverables

- **GitHub repo + README** - how to run it (app, backend, and how to import and run the n8n workflow); the conflict cases you handle; anything you left out and what you'd do next. If you went beyond the core requirements, list what.
- **`DECISIONS.md`** - your data/sync model, your conflict-resolution approach, and a short explanation of **why two devices always end up identical. Explain how idempotency is enforced in both the backend and the n8n workflow. Note one tradeoff** you made and why.
- n8n-workflow.json - the exported workflow(s), importable into a fresh n8n instance.

- **A 5 minute video** - run two clients, take them offline, perform conflicting actions (include an **offline focus session on each device** and a **conflicting task edit**), reconnect, and show them reconcile. Show the n8n notification firing exactly once even though the session synced from both devices. Walk through how your sync works and where it could still break.

Optional extensions (only after the core works)
If you finish the core and want to take it further, any of these are fair game - pick what interests you, you don't need all of them:

- Two-way loop. The student replies to the WhatsApp or notification (done, snooze 10m); an n8n webhook hits the backend, mutates state, and that change reconciles across both devices like any other edit.
- n8n-first, then migrate. Implement the streak/coins reward rule as an n8n workflow first, then show the same logic migrated into Express, and explain the tradeoff in DECISIONS.md. This mirrors how we build at Alcovia: prototype logic in n8n, harden it in the backend as it matures.
- Survives an **app restart / crash** mid-session (state + pending changes persist and still sync).
- Works with **3+ devices**, not just two.
- **Resumes safely** if the network drops in the middle of a sync.
- **Efficient sync** - devices exchange only what changed, not the whole state.
- A **property / fuzz test** that runs many random offline edit sequences across devices and checks they always converge.

- Runs on a **real phone** (Expo Go), not just web.
- Surfaces a conflict to the **user** when an automatic merge isn't obviously right.

What we are really testing
We read DECISIONS.md and watch the full video. We care less about polish and more about judgment: why your merge strategy converges, how you reasoned about idempotency end to end (app to backend to n8n), and your honesty about where it could still break. A clean explanation of a deliberate, simple design beats a fragile clever one.

Good luck - we're excited to see how you approach it. If anything about the scope is genuinely blocking you, reply and ask